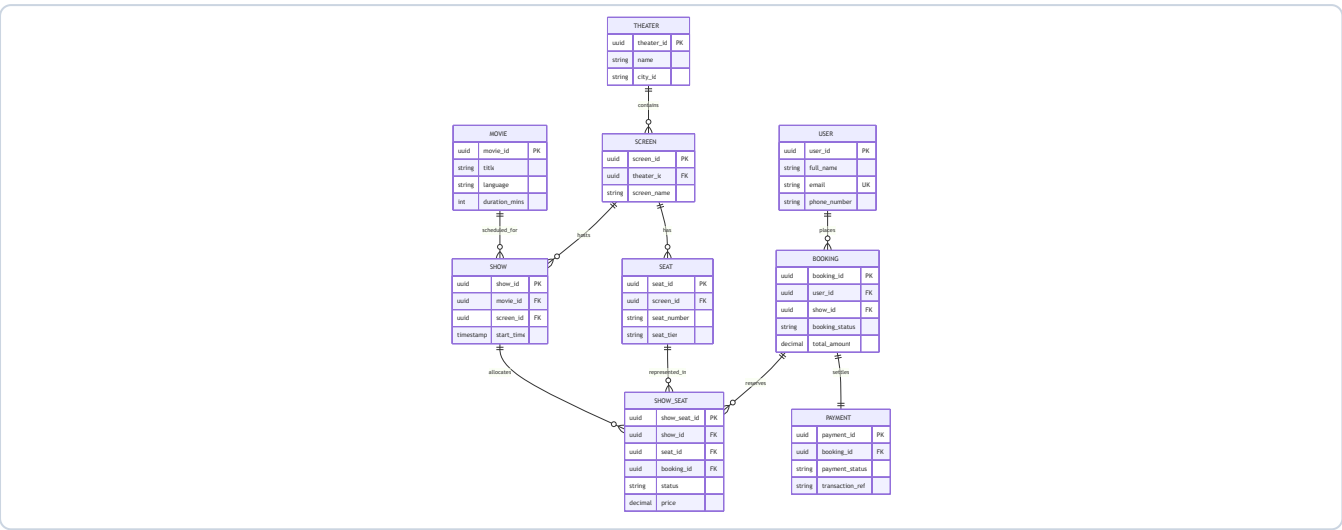


BookMyShow: Entity-Relationship & Storage Model

Complete Architecture Specifications • Section C: Schema Architecture, Storage Types & Indexing

1. Entity-Relationship (ER) Diagram



2. Core Entities & Database Engine Mapping (RDBMS vs. NoSQL)

Entity	Database Type	Keys (PK / FK)	Key Attributes	Lifecycle Enums
SHOW_SEAT	RDBMS (PostgreSQL)	PK: show_seat_id FK: show_id, seat_id, booking_id	price, version (optimistic lock), locked_until	AVAILABLE LOCKED BOOKED
BOOKING	RDBMS (PostgreSQL)	PK: booking_id FK: user_id, show_id	total_amount, created_at, expires_at	PENDING CONFIRMED CANCELLED
PAYMENT	RDBMS (PostgreSQL)	PK: payment_id FK: booking_id	gateway_ref, amount, idempotency_key (UK)	INITIATED SUCCESS FAILED
SHOW / MOVIE / THEATER	RDBMS + Redis Cache	PK: show_id, movie_id FK: screen_id	title, start_time, seat_layout, base_price	SCHEDULED OPEN CLOSED
SEAT_LOCKS (Ephemeral)	NoSQL (In-Memory Redis)	Key: show:seat_id	user_session_id, lock_timestamp (10-min TTL)	ACTIVE_HOLD EXPIRED

3. Maintaining Consistency & Atomicity During Bookings

Atomicity via Multi-Table ACID Transactions:

When payment confirmation arrives, BookingService executes a single atomic transaction across PostgreSQL:

- Updates BOOKING record status from PENDING → CONFIRMED.
- Transitions all associated SHOW_SEAT rows from LOCKED → BOOKED and links the booking_id.
- Records the confirmed PAYMENT row with gateway reference and idempotency key.

If any database operation or network communication fails during this step, the entire transaction rolls back cleanly, leaving zero partial or orphaned records.

Zero Double-Booking Guarantee (Two-Layer Defense):

- **Layer 1 (Fast In-Memory Reservation Lock):**

Distributed Redis locks temporarily hold seats for 10 minutes. If a concurrent user attempts the same seat, the lock acquisition fails immediately, preventing conflicting checkouts at the edge.
- **Layer 2 (Hardware-Enforced Unique Constraint):**

The relational database enforces UNIQUE (show_id, seat_id). Even under extreme edge-case split-brain race conditions, the relational storage engine guarantees that two concurrent transactions can never write or commit identical seats.

Fast Lookups for Available Seats:

To render interactive seating maps instantly, the system pairs a composite index on (show_id, status) with a Redis cached seat bitmap. 95% of seat map queries read directly from memory, while database lookups execute as sub-5ms index-only scans.

4. Strategic Indexing Architecture

Target Table	Index Definition & Columns	Target Query Pattern & Performance Benefit
SHOW_SEAT	idx_showseat_lookup (show_id, status)	Instantly renders real-time auditorium seating layout with seat availability states (sub-5ms index scan).
SHOW_SEAT	uq_show_seat_pair (show_id, seat_id) UNIQUE	Hardware-enforced database constraint preventing duplicate seat allocations for identical showtimes.
MOVIE	idx_movie_city_status (city_id, status, release_date)	Accelerates catalog search filters for movies currently active in a user's selected metropolitan area.
SHOW	idx_show_screen_time (screen_id, start_time)	Prevents overlapping screen schedule conflicts and accelerates theater timetable queries.
BOOKING	idx_booking_user_recent (user_id, created_at DESC)	Optimizes customer profile view and recent booking retrieval with ordered chronological indexing.
PAYMENT	uq_payment_idempotency (idempotency_key) UNIQUE	Guarantees zero duplicate charges from redundant client retries or duplicate gateway webhooks.

5. Sharding & Partitioning Strategy: City + Month Partitioning

- **Horizontal Sharding by City ID:** 99% of user browse and booking traffic is strictly local to a single metropolitan area (e.g., Mumbai, Delhi, Bangalore). Sharding by city_id ensures all queries for theaters, screens, shows, and seats execute strictly on a single database shard, eliminating distributed cross-shard transactions.
- **Time-Based Range Partitioning by Month:** High-churn tables (BOOKING, SHOW_SEAT, PAYMENT) are partitioned by month. Shows older than 30 days are automatically archived to cold storage with zero read/write locks on active partitions.